

Лабораторна робота №4

Розробка вебсерверів засобами фреймворку ASP.NET.

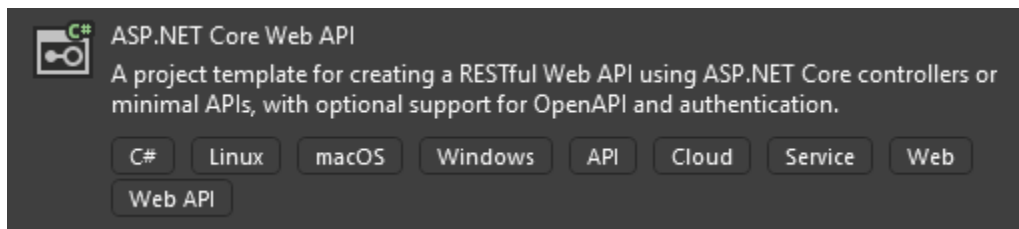
Передумова

Виконайте лабораторну роботу №3. Створіть нову гілку у раніше створеному гіт репозиторії `NUPP_NET_2025_{Номер Групи}_ТК_{Прізвище}_Lab` із назвою **lab4**, яка міститиме код із третьої лабораторної роботи.

Для здачі лабораторної роботи, необхідно буде створити пул реквест із гілки **lab4** у **master**, або якщо на момент здачі пул реквест із гілки lab3 у master не закритий - із гілки lab4 у lab3.

Завдання

1. Створити проєкт за шаблоном [ASP.NET](#) Core Web API {Назва тематики}.REST використовуючи Visual Studio або .NET SDK:
`dotnet new webapi -n {Назва тематики}.REST`



2. Створити папку Models, у якій будуть зберігатися моделі, які будуть використовуватися контролерами у проєкті {Назва тематики}.REST.
3. Створіть контролери у папці Controllers, які будуть описувати REST API для виконання CRUD операцій (створення, читання, оновлення, видалення) над моделями у вашій тематиці, які ви зберігали у базі даних у третій лабораторній, при цьому моделі, що використовуються у контролерах, повинні бути описані у проєкті {Назва тематики}.REST, щоб відповідати [3-рівневій архітектурі](#). Наприклад, якщо у вас є сутність Bus, що зберігається у БД, ви створите новий клас BusModel у папці Models нового проєкту, який використовуватиметься у контролерах. Також зверніть увагу, щоб моделі у контролерах, мали лише необхідні властивості, та якщо якісь властивості зайві, для якихось із методів - створіть нову модель без цих властивостей. **Також у лабораторній роботі повинно бути не менше двох сутностей для яких реалізовані контролери.**
4. При проєктуванні Web API зверніть увагу, щоб створений API відповідав 4-ій умові побудови REST-додатку по Філдингу - однорідності інтерфейсу ([див. 31 слайд](#)), тобто використовувалися унікальні назви сутностей, правильні HTTP методи та поверталися правильні HTTP коди у HTTP відповідях, наприклад 404 якщо сутність не знайдена, або 201 якщо нова сутність створена.
5. Використовуйте асинхронну версію дженерік CRUD сервісу, що використовує репозиторій для доступу до даних та який був реалізований у 3ій лабораторній

роботі, у створених контролерах.:

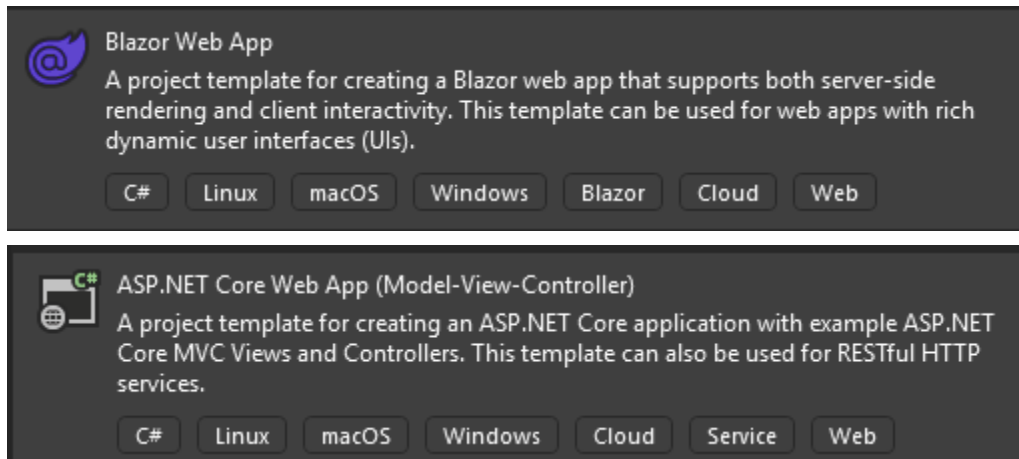
```
public interface ICrudServiceAsync<T>
{
    public Task<bool> CreateAsync(T element);
    public Task<T> ReadAsync(Guid id);
    public Task<IEnumerable<T>> ReadAllAsync();
    public Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
    public Task<bool> UpdateAsync(T element);
    public Task<bool> RemoveAsync(T element);
    public Task<bool> SaveAsync();
}
```

Імплементацию CRUD сервісу, репозиторіїв та контексту, додавайте у контролери використовуючи вбудовану у [ASP.NET](#) Core підтримку впровадження залежностей(Dependency Injection).

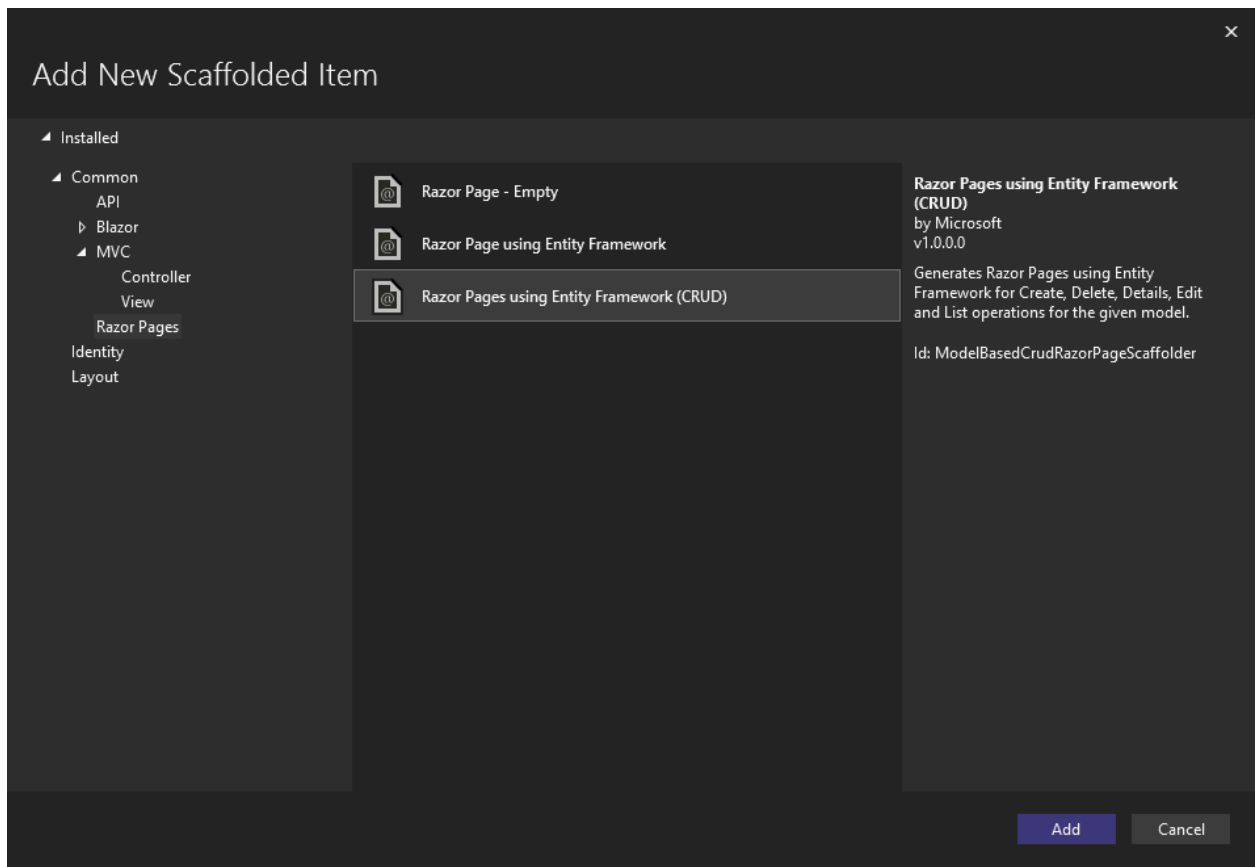
До PR готової лабораторної роботи додайте PDF файл у якому будуть результати виконання запитів, використовуючи створений REST API. Можете скористатися сторінкою Swagger, яка генерується [ASP.NET](#) застосунком, або застосунком [Postman](#).

Додаткове завдання

Створіть проєкт {Назва тематики}.MVC за шаблоном Blazor Web App або [ASP.NET](#) MVC.



У цьому проєкті реалізуйте вебзастосунок, що використовує контекст БД та моделі із 3 лабораторної, використовуючи генерацію сторінок вбудовану у ASP.NET.



Контрольні запитання

1. Які ключові особливості шаблону ASP.NET Core Web API та чим він відрізняється від Blazor Web App або ASP.NET MVC?
2. Поясніть, як реалізується тривірнева архітектура у вашому проєкті {Назва тематики}.REST та які переваги вона має.
3. Навіщо створювати окремі моделі у папці Models та в чому різниця між цими моделями й сутностями, що зберігаються в базі даних?
4. Що таке REST і як у вашому API забезпечується дотримання принципу однорідності інтерфейсу (uniform interface)?
5. Які HTTP-методи використовуються для CRUD-операцій? Наведіть приклади відповідності методів діям у контролерах.

6. Які HTTP-коди відповідей є коректними для сценаріїв успіху та обробки помилок у вашому API (створення, оновлення, видалення, не знайдено тощо)?
7. Опишіть інтерфейс `ICrudServiceAsync<T>`: для чого потрібні асинхронні методи та як вони впливають на продуктивність застосунку?
8. Як працює механізм впровадження залежностей (Dependency Injection) в ASP.NET Core і як ви зареєстрували свої сервіси та репозиторії?
9. Як організована пагінація у вашому CRUD-сервісі (`ReadAllAsync(int page, int amount)`) та для чого вона потрібна?
10. Яким чином ви тестували роботу свого API, які інструменти або сторінки використовували, та які результати следует включити в PDF-файл для PR?
11. Які кроки потрібно виконати, щоб створити проєкт `{Назва тематики}.MVC`, та чим відрізняється його структура від REST API?
12. Як ви повторно використали моделі та контекст бази даних з третьої лабораторної у новому MVC/Blazor-застосунку?

Система оцінювання (ТК)

Усього за лабораторну роботу можна отримати **20 балів**, із яких 14 йдуть у семестровий контроль та 6 у заліковий. Роботи подані через дистанційну систему навчання **до 14 листопада** оцінюються без усного захисту із відповідями на контрольні запитання, при цьому додаткове завдання оцінюється у 4 бали, роботи здані після дедлайну повинні бути захищені згідно контрольних запитань.

Система оцінювання (ТН)

Усього за лабораторну роботу можна отримати **10 балів**. Роботи подані через дистанційну систему навчання **до 15 листопада** оцінюються без усного захисту із відповідями на контрольні запитання, при цьому додаткове завдання оцінюється у 2 бали, роботи здані після дедлайну повинні бути захищені згідно контрольних запитань.